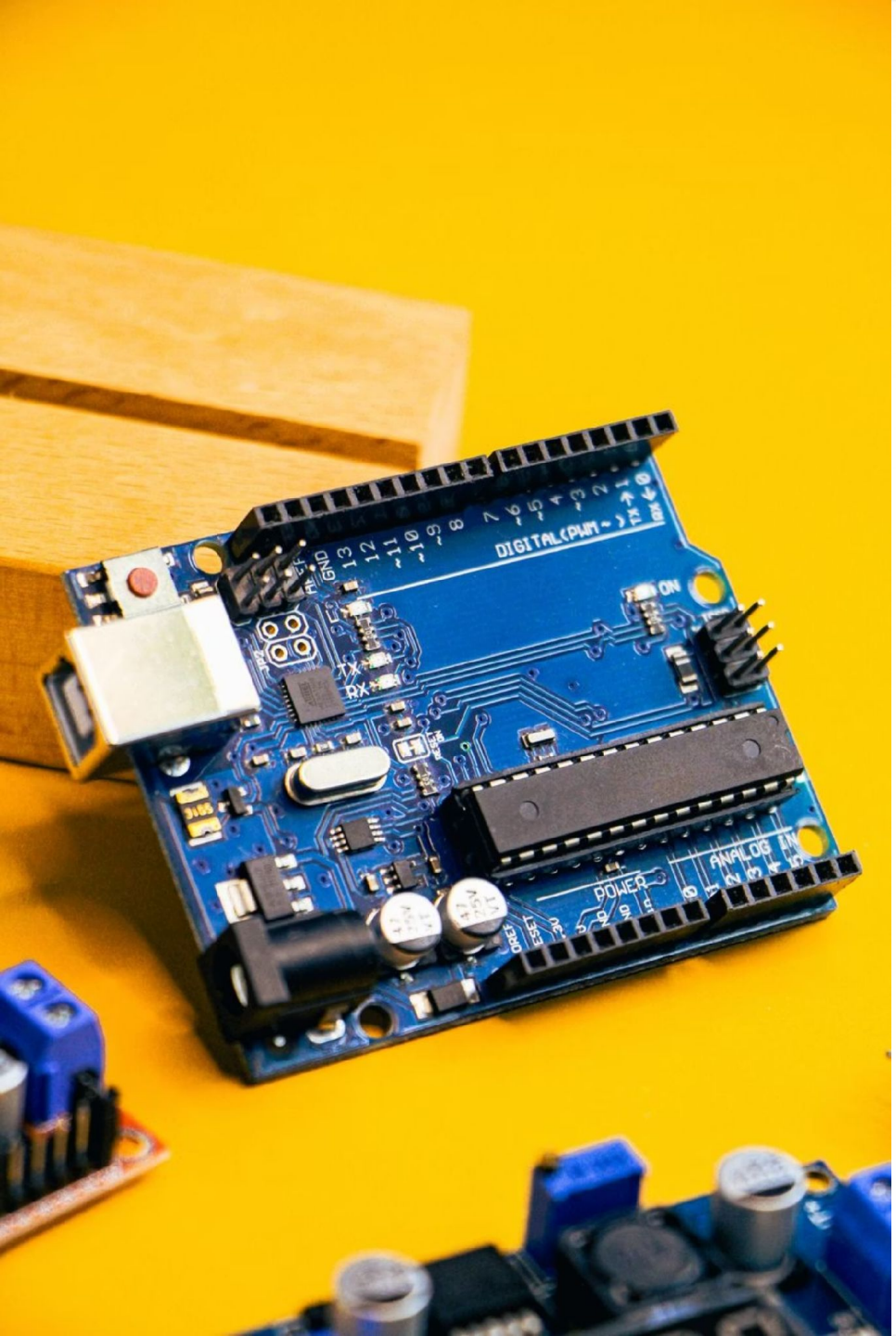


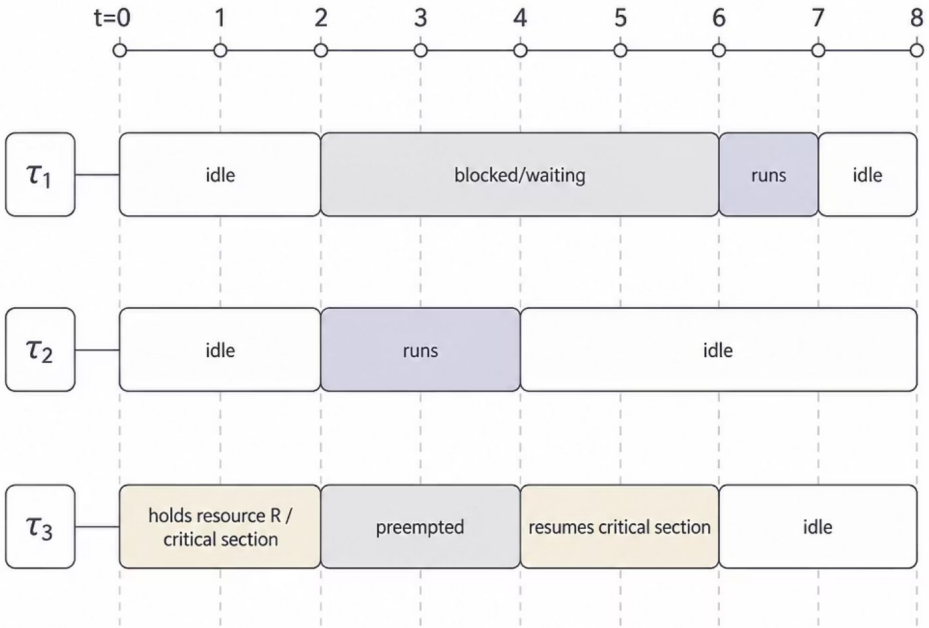
Priority Inheritance Protocol

Resource Access Protocols in Real-Time Systems

Nnaemeka Nnachi-Egwu · Electronic Engineering

Real-Time Systems Seminar · **Prof. Dr. Stefan Henkler**

The Priority Inversion Problem



Scenario: T_3 holds R. T_1 arrives and blocks. T_2 (medium priority, no R) preempts T_3 — T_1 stuck while a **lower-priority** task runs.

Duration: **unbounded** if many medium-priority tasks exist.

⚠ **Kopetz:** A result after its deadline is **logically incorrect**, not just late.

The Fix: Inherit the Blocker's Priority

Before PIP

τ3 holds R at P3; τ1 blocked; τ2 preempts τ3; τ1 stuck unbounded

After PIP

τ3 inherits P1; τ2 cannot preempt; τ3 finishes CS; τ1 unblocked

When τ_1 blocks on S_R held by τ_3, τ_3 **inherits** τ_1's priority:

$$p_j(S_R) = \max\{P_j, \max_h P_h \mid \tau_h \text{ blocked on } S_R\}$$

Effect: τ_2 cannot preempt τ_3 anymore. After signal, τ_3's priority is **restored** to nominal. Blocking bounded to τ_3's remaining CS length.

pi_wait and pi_signal (Buttazzo §7.6.4)

Transitive inheritance propagates the highest blocked priority through the chain.

pi_wait(S)

- If S free → lock it
- Else → block caller, add to queue
- Inherit priority via Eq. (1); apply transitively

pi_signal(S)

- Unlock S
- Wake highest-priority waiter
- Restore holder's priority to max of nominal and remaining waiters

τ3 holds SR and blocks τ2, which blocks τ1; P1 is inherited up.

Theorem 7.2: Bounded Blocking

Sha, Rajkumar & Lehoczky, IEEE Trans. Computers, 1990

Blocking Bound

$$\alpha_i = \min(l_i,\ s_i)$$

l_i = lower-priority tasks that can block τ_i

s_i = semaphores that can block τ_i

$$B_i = \min(B_i^l,\ B_i^s)$$

Complexity: O(mn²). Bound is **not tight** (Buttazzo p. 222).

Schedulability with Blocking

Liu-Layland test (Eq. 7.19):

$$\sum_{h<i} \frac{C_h}{T_h} + \frac{C_i + B_i}{T_i} \leq i\left(2^{1/i} - 1\right)$$

When inconclusive, exact Response-Time Analysis (Eq. 7.22):

$$R_i = C_i + B_i + \sum_{h<i} \left\lceil \frac{R_i}{T_h} \right\rceil C_h$$

Three-Task Example

Task	C	T	D	Prio	CS	B
T_1	1	6	6	1	1	4
T_2	2	8	8	2	—	4
T_3	4	24	24	3	4	0

Shared semaphore *S_R*, ceiling = P_1. $B_1 = 4$ (direct blocking), $B_2 = 4$ (push-through, Lemma 7.1), $B_3 = 0$.

RTA results: $R_1 = 5 \leq 6$ ✓ · $R_2 = 8 \leq 8$ ✓ · $R_3 = 8 \leq 24$ ✓

Liu-Layland inconclusive for T_2 (0.917 > 0.828) — RTA gives exact result.

Five Resource Access Protocols

Buttazzo Table 7.5 — PIP column highlighted

Property	NPP	HLP	PIP	PCP	SRP
Deadlock-free	✓	✓	✗	✓	✓
Chained blocking	—	—	**Yes**	—	—
Transparent	✓	—	✓	—	—
Max blockings	1	1	α_i	1	1
Implementation	Easy	Easy	**Hard**	Medium	Easy
Dynamic priority	—	—	—	—	✓

Formal Verification with UPPAAL

ResourceManager automaton encoding PIP inheritance logic with atomic priority updates.

Model Components

- **Task** template: Idle / Ready / Running / CS / Blocked
- **ResourceManager**: Free / Held / Granting
- Shared: act_prio[3], holder, block_dur[3]

Verified Properties

- P1: `A[] not deadlock`
- P2: `A[](blocking_tau1 ⇒ not tau2.Running)`
- P3: `A[](block_dur ≤ 4)`
- P4: `E<>(act_prio[2]==1)` — inheritance reachable
- P5: `A[](rm.Free ⇒ act_prio[2]==3)` — priority restored

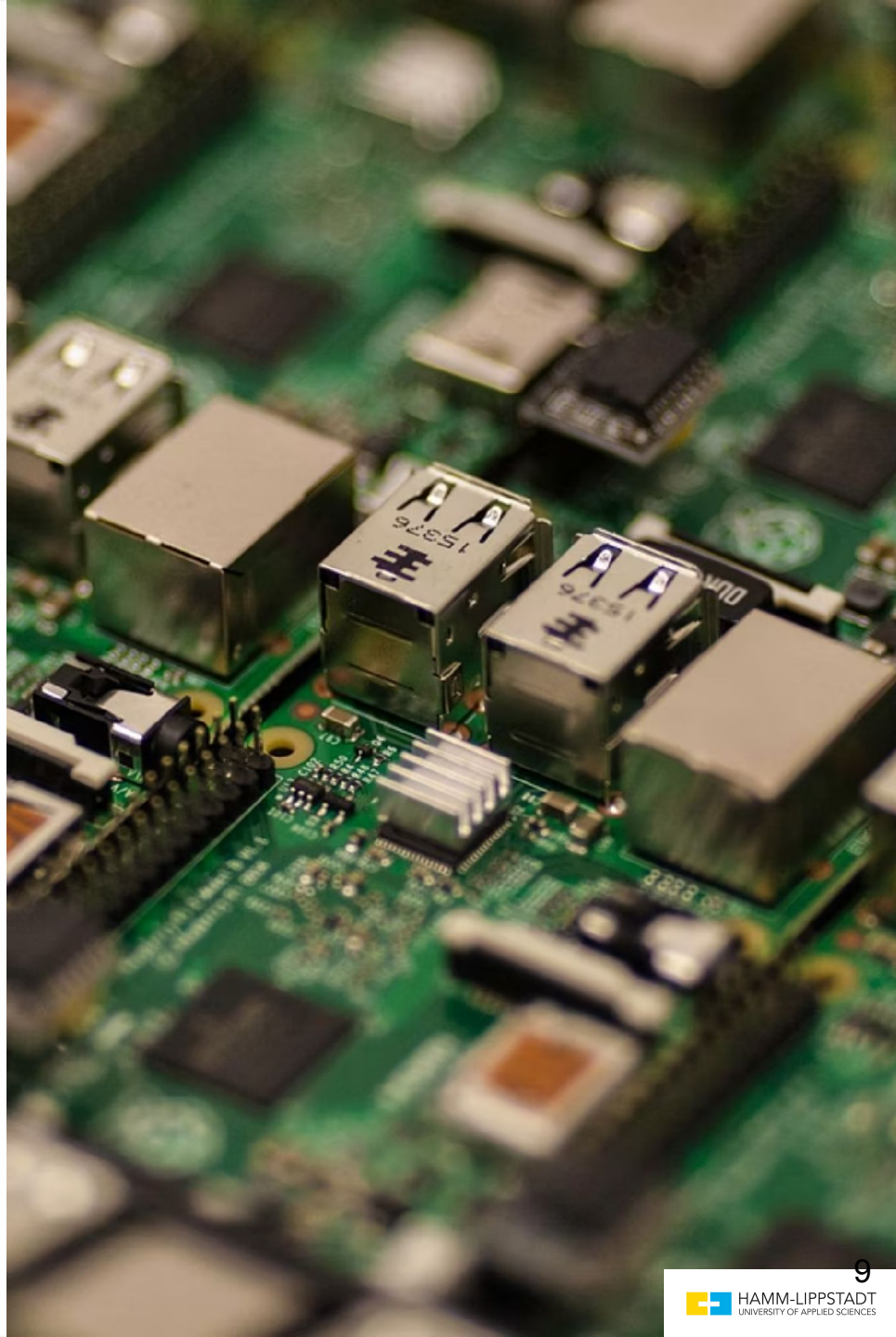
When to Use PIP — and When Not To

✓ Suitable For

- Single-core RTOS
- POSIX legacy migration (`PTHREAD_PRIO_INHERIT`)
- Automotive ECUs with total semaphore ordering
- Mars Pathfinder fix: one config line

✗ Not Suitable For

- Multiprocessors — IPI latency breaks timing (Gracioli 2013)
- EDF scheduling — use SRP instead
- Certified deadlock freedom — use PCP
- WCET sensitivity — B_i only as good as δ_{j,k} inputs (Abella 2015)



Summary

Key Takeaways

1. PIP bounds inversion to $\alpha_i = \min(l_i, s_i)$ CS (Theorem 7.2)
2. Transparency enables deployment without code changes
3. Limitations: no deadlock prevention, chained blocking, hard kernel impl., uniprocessor only

Thank you for your attention, i am open to questions